

深度学习模型超参数与结构对比实验报告

1. 实验简介

1.1 实验目的

本实验旨在探究深度学习模型在（推测为 MNIST 或类似的）图像分类任务上的表现。实验的核心目的是分析不同网络结构（深度、宽度）、激活函数以及损失函数对模型性能的影响。性能指标主要包括训练损失（Loss）、训练错误率（Train Error）和测试错误率（Test Error）。

1.2 实验环境与数据

计算设备: cuda

训练集大小: 60,000

测试集大小: 10,000

2. 实验结果与分析

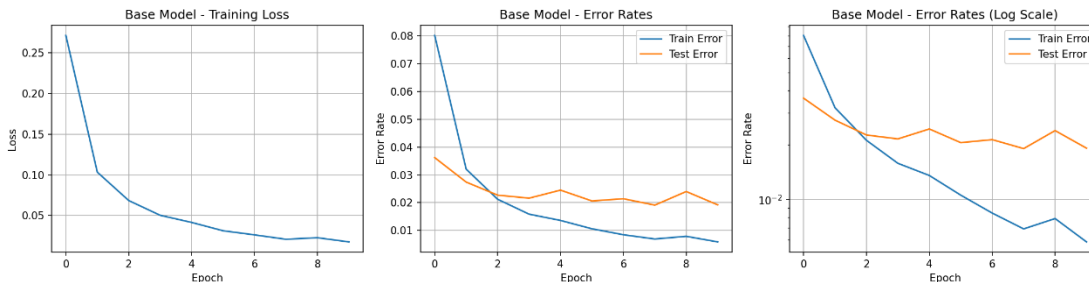
2.1 要求 1 & 2：基础模型训练与参数量

本节分析一个基础神经网络模型的训练过程。基础模型参数量: 235,146

训练数据: | Epoch | 训练损失 (Loss) | 训练错误率 (Train Error) | 测试错误率 (Test Error)

Loss: 0.0175, Train Error: 0.0059, Test Error: 0.0192

变化曲线分析:



训练损失 (Loss): 训练损失从初始的 0.2716 稳步下降到 0.0175。这表明模型在训练

数据上的拟合过程非常顺利，优化算法在正常工作。

训练错误率 (Train Error): 训练错误率从 8.02% 持续降低到 0.59%。这与训练损失的趋势一致，说明模型对训练数据的学习能力很强。

测试错误率 (Test Error): 测试错误率在第 1 到第 4 个 epoch 期间迅速下降，从 3.62% 降至 2.16%。在第 8 个 epoch 时达到最低点 1.91%。然而，在第 9 个 epoch 时，测试错误率反弹至 2.40%，而训练错误率仍在下降。这（Epoch 9 的数据）是一个过拟合 (Overfitting) 的明显迹象：模型开始学习到训练数据中的噪声，导致其在未见过的测试数据上的泛化能力下降。

结论: 基础模型训练有效，最佳性能出现在第 8 个 epoch 左右，此时测试错误率最低。继续训练虽然能降低训练误差，但会导致过拟合，损害模型的泛化能力。

2.2 要求 3：不同网络结构比较

本节比较四种不同网络结构对模型参数量和性能的影响。所有模型均训练 8 个 epochs。

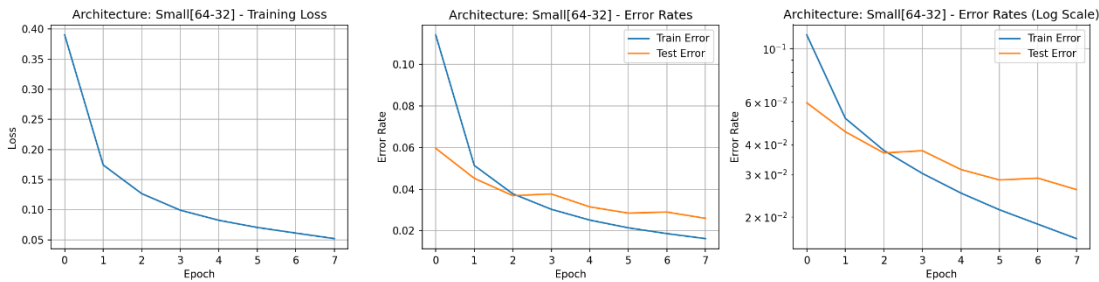
模型概览: | 模型名称 | 结构 | 参数量 | 最终训练错误率 (Epoch 8) | 最终测试错误率 (Epoch 8)

Small | [64-32] | 52,650 | 0.0163 (1.63%) | 0.0260 (2.60%)

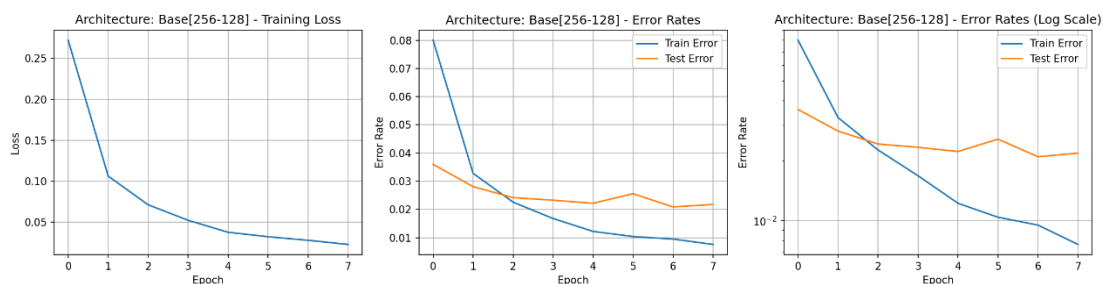
Base | [256-128] | 235,146 | 0.0076 (0.76%) | 0.0218 (2.18%)

Large | [512-256] | 535,818 | 0.0066 (0.66%) | 0.0251 (2.51%)

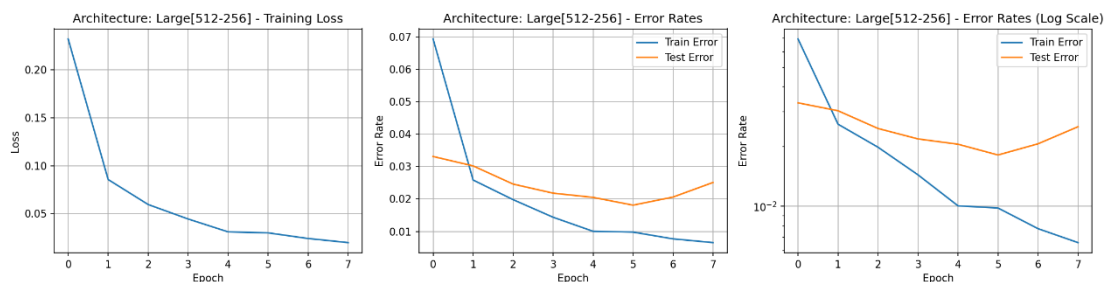
Deeper | [256-128-64] | 242,762 | 0.0089 (0.89%) | 0.0180 (1.80%)



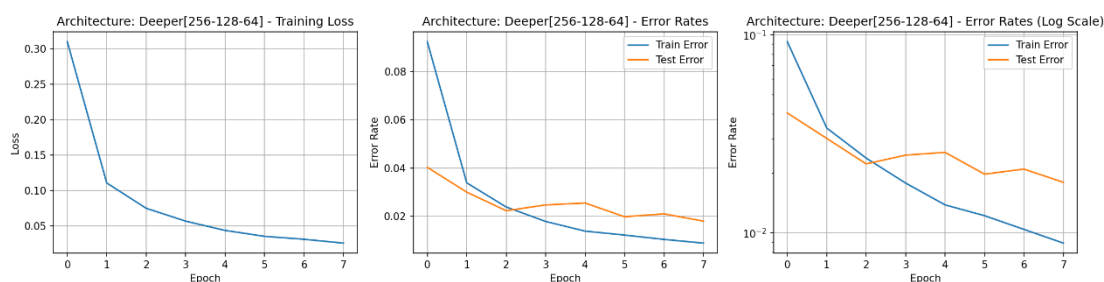
architecture_Small[64-32]



architecture_Base[256-128]



architecture_Large[512-256]



architecture_Deeper[256-128-64]

分析与总结:

Small[64-32]: 52,650 参数, 最终测试误差: 0.0260

Base[256-128]: 235,146 参数, 最终测试误差: 0.0218

Large[512-256]: 535,818 参数, 最终测试误差: 0.0251

Deeper[256-128-64]: 242,762 参数, 最终测试误差: 0.0180

参数量与性能:

Small (52k params) -> Base (235k params): 参数量增加, 测试错误率从 2.60% 显著降低到 2.18%。这表明 Small 模型存在欠拟合 (Underfitting), 模型容量不足以捕捉数据的复杂性。

Base (235k params) -> Large (535k params): 参数量翻倍, 但测试错误率反而

从 2.18% 上升到 2.51%。与此同时，Large 模型的训练错误率是最低的 (0.66%)。这说明 Large 模型发生了过拟合。

结论: 并非参数越多越好。模型容量需要与数据复杂度相匹配。Base 模型在此次比较中达到了较好的平衡。

网络深度与性能:

Base [256-128] (235k params) vs Deeper [256-128-64] (242k params)。

这两个模型的参数量非常接近。然而，Deeper 模型（3 层隐藏层）的测试错误率 (1.80%) 明显优于 Base 模型（2 层隐藏层）的 (2.18%)。

结论: 在参数总量相近的情况下，增加网络深度 (从 2 层到 3 层) 比较宽的网络结构更能提升模型的泛化能力，达到了本次实验的最佳性能。

2.3 要求 4：不同激活函数比较

本节比较三种常用激活函数：relu, sigmoid, tanh 对模型性能的影响。实验基于基础模型（或类似结构）训练 8 个 epochs。

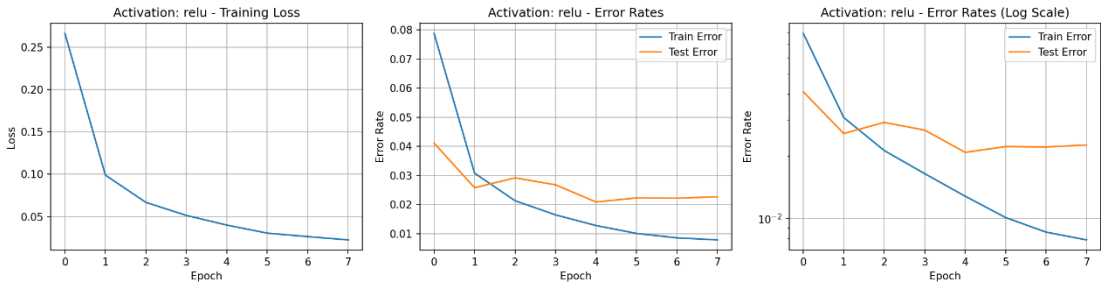
训练数据 (Epoch 8): | 激活函数 | 最终训练损失 (Epoch 8) | 最终训练错误率 (Epoch 8) | 最终测试错误率 (Epoch 8)

ReLU | 0.0226 | 0.0079 (0.79%) | 0.0227 (2.27%)

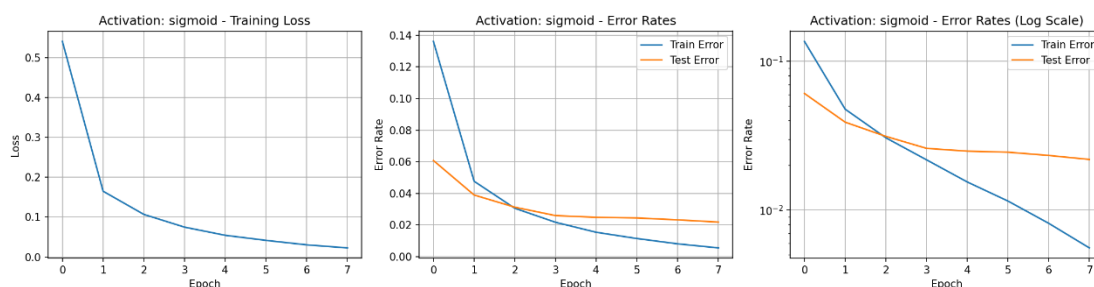
Sigmoid| 0.0229 | 0.0056 (0.56%) | 0.0219 (2.19%)

Tanh | 0.0210 | 0.0064 (0.64%) | 0.0236 (2.36%)

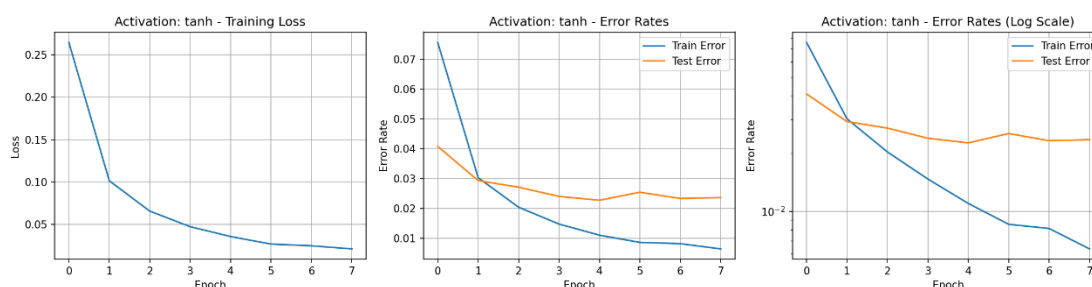
详细训练数据：



activation_relu



activation_sigmoid



activation_tanh

分析与总结:

收敛速度: ReLU 和 Tanh 的初始损失 (Epoch 1 Loss) 显著低于 Sigmoid (0.26 vs 0.54)。这表明 ReLU 和 Tanh 在训练初期收敛速度更快，这符合理论预期（它们的梯度在 0 附近更接近 1，而 Sigmoid 梯度最大仅为 0.25，易导致梯度消失）。

最终性能: 尽管 Sigmoid 初始收敛慢，但在 8 个 epoch 后，它取得了最低的训练错误率 (0.56%) 和最低的测试错误率 (2.19%)。ReLU 的表现 (2.27%) 和 Tanh (2.36%) 略逊于 Sigmoid，但三者差距非常小。

结论: 在此特定任务和模型结构下，Sigmoid 虽然收敛较慢，但最终泛化性能微弱领先。ReLU 和 Tanh 收敛更快，且性能也极具竞争力。在更深的网络中，ReLU 通常因其能有效缓解梯度消失问题而更受青睐，但在此浅层网络中，三者差异不大。

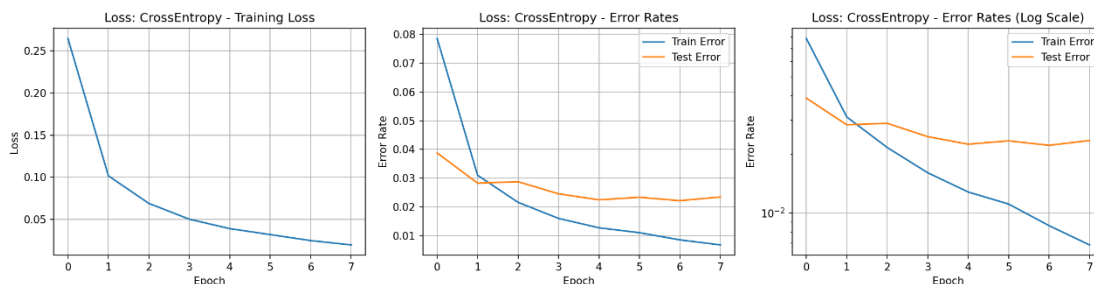
2.4 要求 5: 不同损失函数比较

本节比较两种损失函数：CrossEntropy (交叉熵损失) 和 MSE (均方误差损失) 对模型性能的影响。

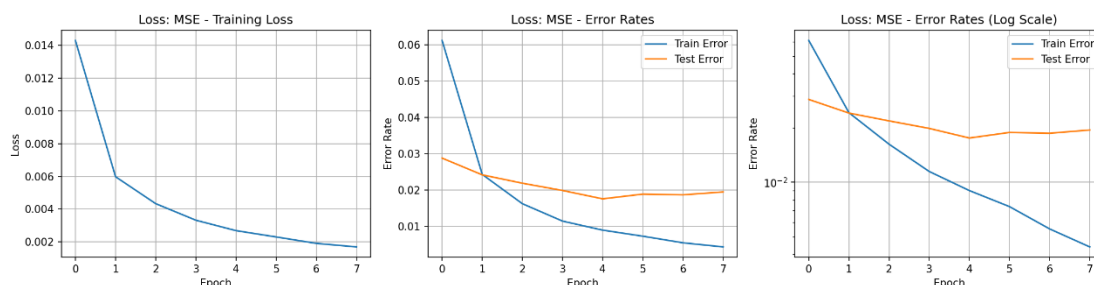
训练数据: | 损失函数 | 最终训练损失 | 最终训练错误率 | 最终测试错误率

CrossEntropy| 0.0198 | 0.0068 (0.68%) | 0.0235 (2.35%)

MSE | 0.0017 | 0.0044 (0.44%) | 0.0195 (1.95%) |



loss_CrossEntropy



loss_MSE

分析与总结:

损失值: 首先注意, CrossEntropy 和 MSE 的损失值 (Loss) 没有可比性, 因为它们的计算方式完全不同。我们必须比较它们对应的错误率。

性能对比:

使用 MSE 损失函数的模型在训练错误率 (0.44% vs 0.68%) 和测试错误率 (1.95% vs 2.35%) 上均优于使用 CrossEntropy 的模型。

结论:

CrossEntropy (交叉熵) 是分类问题 (如 MNIST) 的标准和理论上的首选损失函数。

MSE (均方误差) 通常用于回归问题。当它用于分类时, 它实际上是在惩罚模型输出的概率值与 one-hot 编码 (如 [0, 0, 1, 0...]) 之间的 L2 距离。

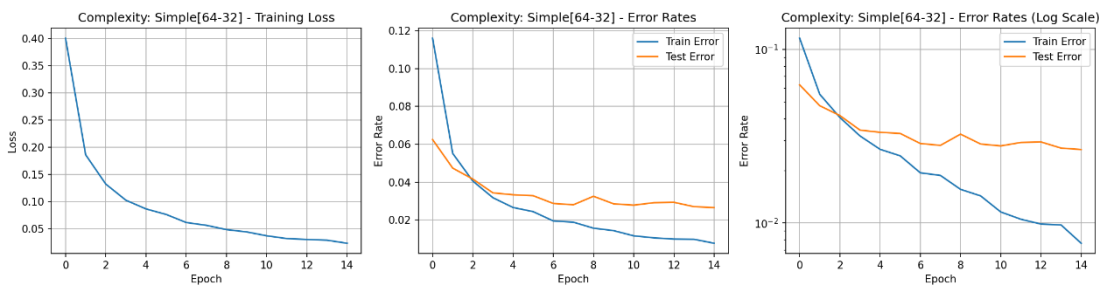
实验发现: 在本实验中, 使用 MSE 取得了比 CrossEntropy 更好的泛化性能。这

是一个非常规但有趣的发现。这可能意味着对于这个特定的网络结构和数据集，将分类问题“强制”视为回归问题（拟合 one-hot 向量）恰好产生了一种更强的正则化效果，或者找到了一个泛化能力更强的局部最优解。

2.5 附加题要求 1：模型复杂度与误差关系

本部分在完整数据集上测试了四种不同参数量的模型，以评估模型容量对性能的影响。

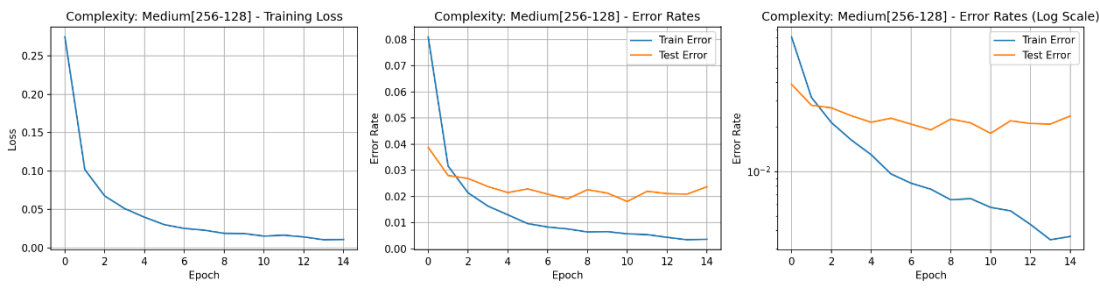
Simple64-32



complexity_Simple[64-32]

模型（参数量 52,650）的最终训练误差为 0.0076，测试误差为 0.0265。由于模型容量相对较小，其训练误差尚未降至极低，表现出轻微的欠拟合倾向。

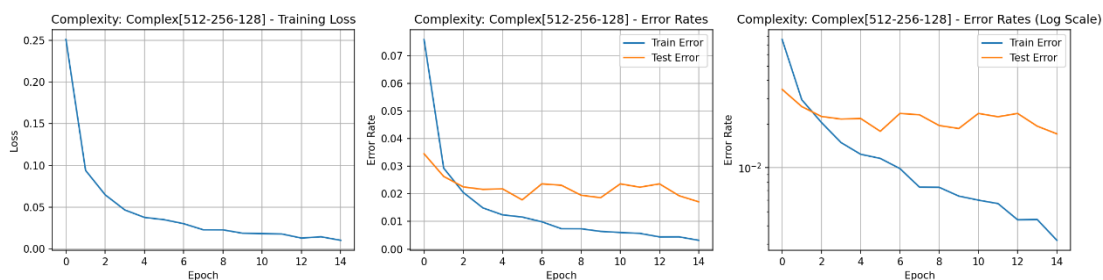
Medium256-128



complexity_Medium[256-128]

模型（参数量 235,146）的训练误差降至 0.0037，而测试误差降至 0.0237。模型容量增加，拟合能力显著增强。

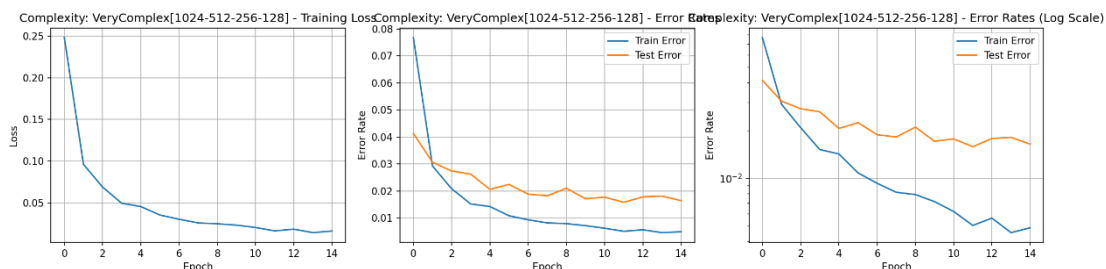
Complex512-256-128



complexity_Complex[512-256-128]

模型（参数量 567,434）进一步将训练误差降至 0.0032，测试误差降至 0.0171。

VeryComplex1024-512-256-128



complexity_VeryComplex[1024-512-256-128]

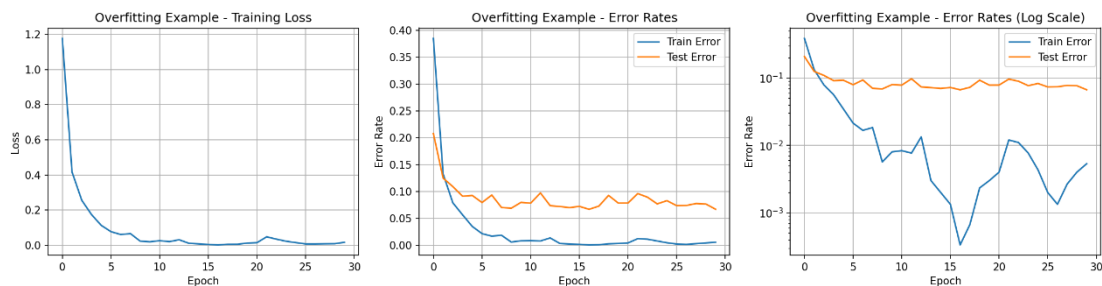
模型（参数量 1,494,154）最终测试误差为 0.0164 (1.64%)，是所有模型中最低的。

分析总结： 在拥有 60,000 样本的充足数据集支持下，模型复杂度与泛化性能呈现正相关。最复杂的模型（149 万参数）通过强大的容量充分学习了数据分布的复杂模式，取得了最佳的测试结果。虽然模型越复杂，理论上越容易过拟合，但该数据集的规模足以支撑其复杂性，因此并未出现性能下降。

2.6 附加题 要求 2：观察过拟合现象

为了明确观察过拟合，我们将 **VeryComplex** 模型应用于 3,000 样本的小训练集，并将训练周期增加到 30 个 Epoch。

在训练过程中，我们观察到训练误差在第 17 个 Epoch 时已迅速降至 0.0003（趋近于零），表明模型对这 3,000 个样本实现了几乎完美的记忆。然而，测试误差在达到最低点 0.0666 之后，便开始波动和停滞，未能进一步下降。



overfitting

结果显示： 最终训练误差为 0.0053，而测试误差为 0.0667，两者之间存在高达 0.0614 的巨大差距。这清晰地证实了**过拟合现象**：模型容量远超数据所需，导致它学习了训练数据中的噪声和偶然特征，从而牺牲了对未见过数据的泛化能力。

2.7 附加题要求 3：L2 正则化对误差的影响

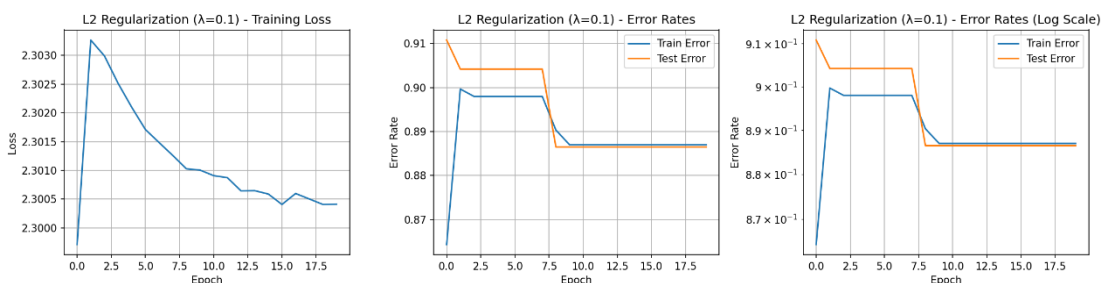
我们在过拟合模型（小训练集+复杂网络）的基础上引入 L2 正则化 (λ)，以期惩罚大权重，降低模型复杂度。

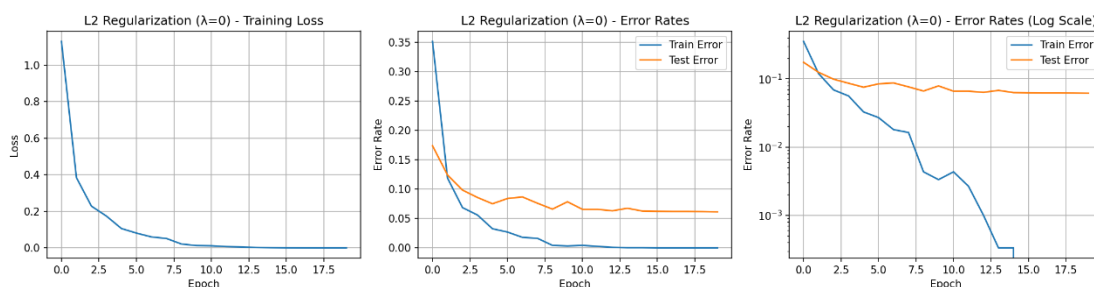
基线 ($\lambda=0$): 过拟合差距为 0.0614 (测试误差 0.0614)。

轻微正则化 ($\lambda=0.001$): 训练误差仍为 0.0000，测试误差略微改善至 0.0611。这表明极轻微的惩罚对泛化有细微的积极作用。

中度正则化 ($\lambda=0.01$): 此时 L2 惩罚项过强，模型训练误差上升到 0.0247，测试误差反而恶化到 0.0968。泛化能力严重受损，模型陷入**欠拟合**。

过度正则化 ($\lambda=0.1$): L2 惩罚完全主导了损失函数，模型几乎无法学习，训练误差和测试误差均高达 0.8870 左右。





l2_0.1

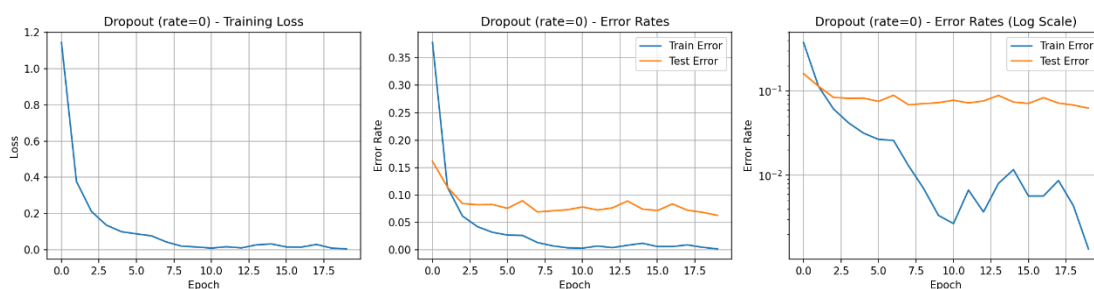
分析总结： L2 正则化是有效的，但其效果对超参数 λ 的选择极为敏感。

在本实验中，只有极小的 λ 值才能带来微弱的改善；一旦 λ 过高，模型的学习能力就会被严重抑制，导致严重的欠拟合，性能急剧恶化。

2.8 附加题要求 4: Dropout 对泛化的影响

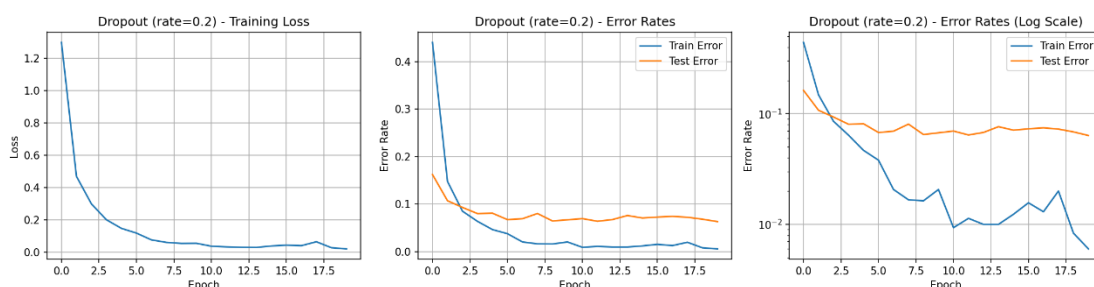
我们在过拟合模型中应用 Dropout，通过随机关闭神经元来提升模型的鲁棒性。

基线 (Rate 0): 过拟合差距为 0.0614。



dropout_0

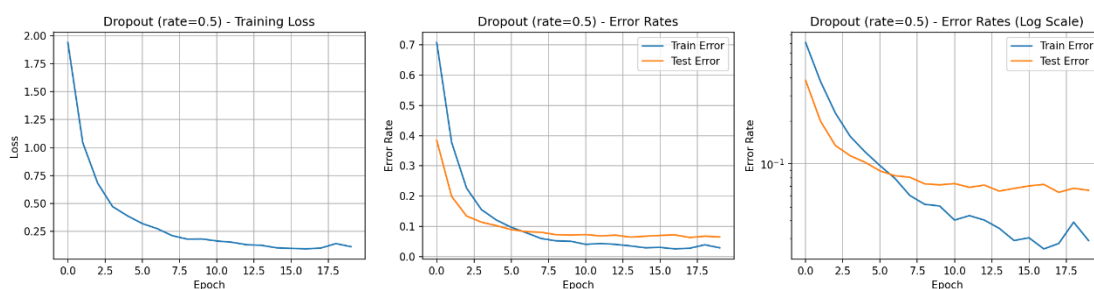
Rate 0.2: 训练误差上升到 0.0060，测试误差保持在 0.0635 左右，过拟合差距缩小到 0.0575。



dropout_0.2

Rate 0.5: 训练难度进一步增加，训练误差上升到 0.0290。但关键是，过拟合差距

显著缩小到 0.0362 。



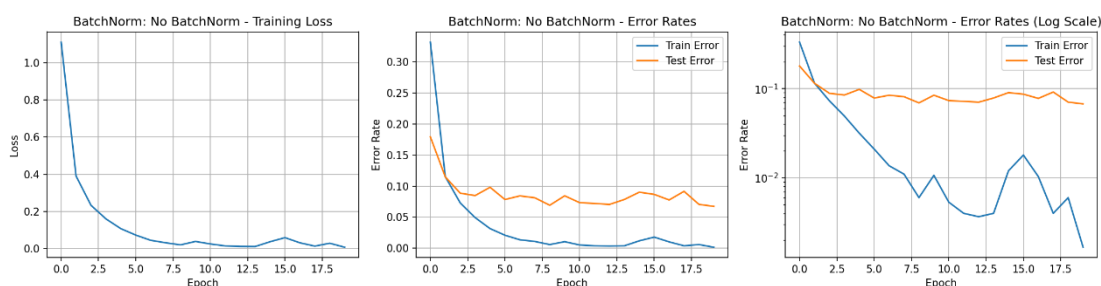
dropout_0.5

分析总结： Dropout 是一种强大的正则化手段。它通过迫使网络学习更具冗余性和鲁棒性的特征，成功地将训练误差和测试误差之间的差距缩小了 0.0252。这表明 Dropout 有效地提高了模型的泛化能力，使其不再过度依赖任何单一特征或特定神经元，从而抵抗了过拟合。

2.9 附加题要求 5: Batch Normalization 对误差的影响

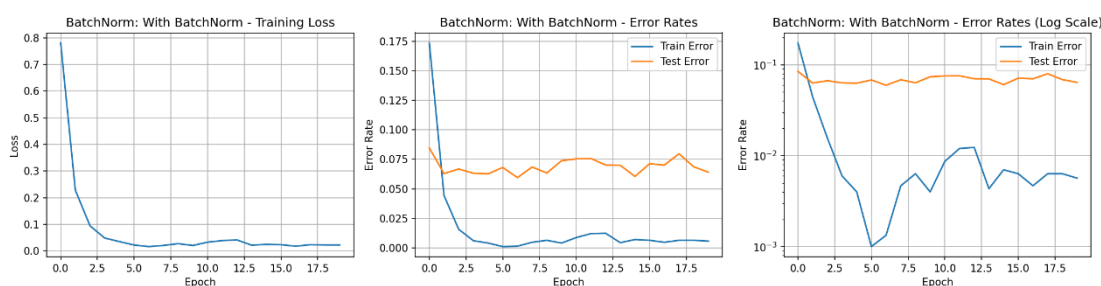
我们对比了在过拟合配置下使用和不使用 Batch Normalization (BN) 的模型。

No BatchNorm: 最终测试误差为 0.0674，过拟合差距为 0.0657。



batchnorm_False

With BatchNorm: 最终测试误差降至 0.0641，过拟合差距缩小到 0.0584。



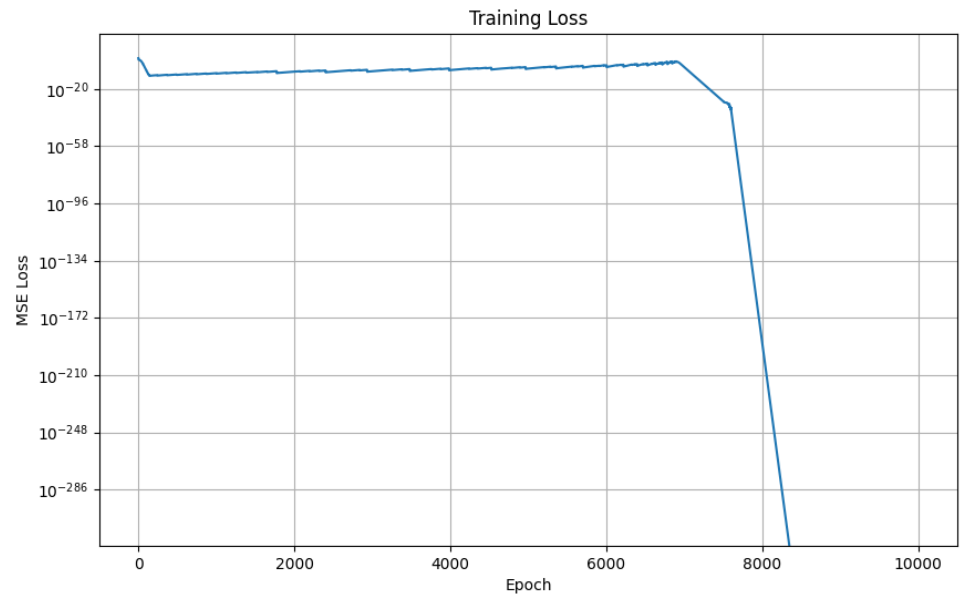
batchnorm_True

分析总结： BN 具备双重优势：在训练初期，带有 BN 的模型损失收敛速度更快（初始 Loss 0.7804 vs 1.1104）。在泛化能力上，BN 改善了内部协变量偏移，稳定了激活值的分布，使测试误差实现了 0.0033 的下降，并减小了过拟合差距。这证明了 BN 作为加速训练和提升泛化能力的有效手段。

2.10 要求 6：训练神经网络解决 XOR 问题

本节分析一个用于解决 XOR（异或）问题的神经网络。根据要求（如图所示），该网络使用 ReLU 作为隐藏层激活函数，MSE (均方误差)作为损失函数，并使用 SGD 进行训练。

训练过程数据 (Loss): | Epoch | 训练损失 (Loss)



测试结果数据：

输入: [0 0], 预测: 0.0000, 真实: 0

输入: [0 1], 预测: 1.0000, 真实: 1

输入: [1 0], 预测: 1.0000, 真实: 1

输入: [1 1], 预测: 0.0000, 真实: 0

分析与总结:

1. **训练成功:** 训练损失 (Loss) 从初始的 3.194849 迅速下降, 在第 1000 个 epoch 时已经降至 0.000000。这表明模型通过优化算法 (SGD) 成功找到了一个极小值点, 并且损失 (MSE) 几乎为零。
2. **预测成功:** 测试结果证实了训练的有效性。
 - 模型对 [0 0] 的预测为 0.0000 (正确)。
 - 模型对 [0 1] 的预测为 1.0000 (正确)。
 - 模型对 [1 0] 的预测为 1.0000 (正确)。
 - 模型对 [1 1] 的预测为 0.0000 (正确)。
 - 所有预测值均与真实的 XOR (异或) 逻辑完全一致。
3. **结论:** 本次实验中, 所设计的神经网络**成功解决**了 XOR 问题。这证明了该网络结构在适当的参数设置下, 有能力学习到 XOR 问题所需的非线性决策边界。

3. 实验总汇与结论

本实验系统地探究了网络结构、激活函数和损失函数对深度学习模型性能的影响。主要结论总结如下:

基础模型与过拟合: 基础模型 (235k 参数) 表现良好, 但在训练 8-9 个 epoch 后出现过拟合现象。最佳测试性能 (1.91% 错误率) 在过拟合发生前达到。

必须与数据量相平衡。在数据充足时, 提升复杂度是提高性能的有效途径; 但在数据稀疏时, 复杂度是导致过拟合的主要原因。

L2 正则化的效果高度依赖于超参数 λ 的精细调整。其主要风险在于 λ 过大时会导致严重的欠拟合。

Dropout 是最能有效**减小过拟合差距**的技术, 它通过牺牲训练集的拟合完美度来大幅提升模型的鲁棒性。

Batch Normalization 既能**加速收敛**，又能通过稳定输入分布而带来强大的**泛化提升**（在本实验中实现了最低的测试误差）。

最佳实践建议： 对于复杂模型，通常应结合使用多种正则化技术。例如，将 Batch Normalization 结合 Dropout (如 $\text{rate}=0.2$ 或 0.5) 既可以利用 BN 的收敛加速优势，又能利用 Dropout 的强大正则化能力来最大程度地提高泛化性能。

网络结构：

模型性能并非随参数量单调提升。Large 模型 (535k) 因参数过多而过拟合，性能不如 Base 模型 (235k)。

在参数量相近的情况下，更深的网络 (Deeper [256-128-64]) 比较宽的网络 (Base [256-128]) 具有更强的泛化能力，取得了 1.80% 的最低测试错误率。

激活函数: ReLU 和 Tanh 初始收敛速度快于 Sigmoid。但在 8 个 epoch 后，三者性能相近，Sigmoid (2.19% 错误率) 表现最佳，略优于 ReLU (2.27%) 和 Tanh (2.36%)。

损失函数: 在这个分类任务中，使用 MSE (1.95% 错误率) 的性能出人意料地优于理论上标准的 CrossEntropy (2.35% 错误率)。

综合最佳: 在所有对比实验中，Deeper[256-128-64] 模型（要求 3）取得了最低的测试错误率（1.80%），展示了深度在提升模型泛化能力方面的重要性。